

LLM Agents & Deep Q-Learning with Atari Games Assignment

Game: Demon Attack

Baseline Performance

```
print("\n Training baseline agent...")
baseline_agent = DQNAgent(
    state_shape=state_shape,
    n_actions=n_actions,
    learning_rate=0.7,
    gamma=0.8,
    epsilon=1.0,
    epsilon_min=0.01,
    epsilon_decay=0.01
)

baseline_rewards, baseline_steps, baseline_epsilons = train_agent(
    baseline_agent,
    episodes=5000,
    max_steps=99,
    log_freq=100
)
```

BASELINE RESULTS:

Final Avg Reward: 25.00

Final Avg Steps: 99.00

Max Reward: 60.00

The agent failed to learn. It was stuck at 99 steps.

Environment Analysis

STATES:

- State Space: 4 stacked grayscale frames of 84×84 pixels
- Total dimensions: $4 \times 84 \times 84 = 28,224$ input values
- Preprocessing: RGB $\rightarrow$ Grayscale $\rightarrow$ Resize $\rightarrow$ Normalize (0-1)
- Frame stacking provides temporal information (motion)

ACTIONS:

- Action Space: 6 discrete actions
- Actions: ['NOOP', 'FIRE', 'RIGHT', 'LEFT', 'RIGHTFIRE', 'LEFTFIRE']

Q-TABLE SIZE:

- Traditional Q-table: IMPOSSIBLE (continuous/huge state space)
- Solution: Deep Q-Network (DQN) approximates Q-values
- DQN Architecture:
 - * Conv layers: Extract spatial features from frames
 - * FC layers: Map features to Q-values for each action
 - * Output: 6 Q-values (one per action)

ENVIRONMENT ANALYSIS:

States: $4 \times 84 \times 84$ stacked grayscale frames

State dimensions: 28224 pixels

Actions: 6 discrete actions

Action meanings: ['NOOP', 'FIRE', 'RIGHT', 'LEFT', 'RIGHTFIRE', 'LEFTFIRE']

Solution: Using Deep Q-Network to approximate Q-values

Reward Structure

REWARDS:

- Using Atari's native reward system
- Positive rewards for: Destroying enemies, surviving longer
- Negative rewards/penalties: Getting hit, losing lives
- Reward range observed: 0 to 60+ per episode

WHY THIS REWARD STRUCTURE:

1. Built-in rewards align with game objectives
2. Sparse rewards make learning challenging (realistic RL scenario)
3. Encourages agent to:
 - Learn survival strategies
 - Develop offensive tactics (destroy enemies)
 - Balance exploration vs exploitation
4. No reward shaping needed - tests pure RL capability

Bellman Equation Parameters

ALPHA (Learning Rate) SELECTION:

Initial Choice: $\alpha = 0.7$

- Rationale: Standard high learning rate for Q-learning
- Result: FAILED - Agent stuck at 99 steps, reward = 25

Tested Alternatives:

1. $\alpha = 0.001$

- Result:

```
Baseline ( $\alpha=0.7$ ,  $\gamma=0.8$ ):
  Final Avg Reward: 25.0
  Final Avg Steps: 99.0
  Max Reward: 60.0
  Agent failed to learn (stuck at 99 steps)

=====
TESTING ALTERNATIVE ALPHA VALUES (1000 episodes each)
=====

=====
Testing Alpha = 0.001 ( $\gamma=0.8$  fixed)
=====

[W1106 21:45:27.484284944 NNPack.cpp:57] Could not initialize NNPack! Reason: Unsupported hardware.
[W1106 21:45:27.658602894 NNPack.cpp:57] Could not initialize NNPack! Reason: Unsupported hardware.
Training for 1000 episodes...
  Episode 0: Avg Reward = 70.00, Avg Steps = 500.0,  $\epsilon = 0.9900$ 
  Episode 100: Avg Reward = 80.90, Avg Steps = 484.2,  $\epsilon = 0.3624$ 
  Episode 200: Avg Reward = 78.60, Avg Steps = 474.2,  $\epsilon = 0.1326$ 
  Episode 300: Avg Reward = 85.20, Avg Steps = 461.0,  $\epsilon = 0.0486$ 
  Episode 400: Avg Reward = 77.50, Avg Steps = 474.9,  $\epsilon = 0.0178$ 
  Episode 500: Avg Reward = 75.80, Avg Steps = 480.7,  $\epsilon = 0.0100$ 
  Episode 600: Avg Reward = 71.00, Avg Steps = 464.5,  $\epsilon = 0.0100$ 
  Episode 700: Avg Reward = 59.00, Avg Steps = 457.3,  $\epsilon = 0.0100$ 
  Episode 800: Avg Reward = 61.10, Avg Steps = 443.8,  $\epsilon = 0.0100$ 
  Episode 900: Avg Reward = 70.60, Avg Steps = 486.4,  $\epsilon = 0.0100$ 

Results for  $\alpha=0.001$ :
  Final Avg Reward: 76.20 (Baseline: 25.00)
  Final Avg Steps: 479.8 (Baseline: 99.0)
  Max Reward: 160.00
  Improvement: +51.20 reward, +380.8 steps
```

- Average Reward: 76.20

- Average Steps: 479.8 / 500

2. $\alpha = 0.0005$

- Result:

```
=====
Testing Alpha = 0.0005 ( $\gamma=0.8$  fixed)
=====

Training for 1000 episodes...
[W1106 22:18:55.840374678 NNPack.cpp:57] Could not initialize NNPack! Reason: Unsupported hardware.
[W1106 22:18:55.847323754 NNPack.cpp:57] Could not initialize NNPack! Reason: Unsupported hardware.
  Episode 0: Avg Reward = 120.00, Avg Steps = 488.0,  $\epsilon = 0.9900$ 
  Episode 100: Avg Reward = 77.60, Avg Steps = 465.9,  $\epsilon = 0.3624$ 
  Episode 200: Avg Reward = 78.40, Avg Steps = 476.8,  $\epsilon = 0.1326$ 
  Episode 300: Avg Reward = 78.10, Avg Steps = 476.0,  $\epsilon = 0.0486$ 
  Episode 400: Avg Reward = 76.90, Avg Steps = 460.9,  $\epsilon = 0.0178$ 
  Episode 500: Avg Reward = 84.20, Avg Steps = 453.1,  $\epsilon = 0.0100$ 
  Episode 600: Avg Reward = 80.10, Avg Steps = 486.8,  $\epsilon = 0.0100$ 
  Episode 700: Avg Reward = 78.40, Avg Steps = 487.5,  $\epsilon = 0.0100$ 
  Episode 800: Avg Reward = 77.90, Avg Steps = 476.9,  $\epsilon = 0.0100$ 
  Episode 900: Avg Reward = 75.00, Avg Steps = 480.4,  $\epsilon = 0.0100$ 

Results for  $\alpha=0.0005$ :
  Final Avg Reward: 80.60 (Baseline: 25.00)
  Final Avg Steps: 474.2 (Baseline: 99.0)
  Max Reward: 160.00
  Improvement: +55.60 reward, +375.2 steps
```

3. $\alpha = 0.0001$

- Result: Best performance

```
=====
Testing Alpha = 0.0001 ( $\gamma=0.8$  fixed)
=====
Training for 1000 episodes...
[W1106 22:52:42.152531429 NNPACK.cpp:57] Could not initialize NNPACK! Reason: Unsupported
[W1106 22:52:42.159417841 NNPACK.cpp:57] Could not initialize NNPACK! Reason: Unsupported
Episode 0: Avg Reward = 80.00, Avg Steps = 500.0,  $\epsilon = 0.9900$ 
Episode 100: Avg Reward = 86.80, Avg Steps = 481.1,  $\epsilon = 0.3624$ 
Episode 200: Avg Reward = 80.80, Avg Steps = 452.5,  $\epsilon = 0.1326$ 
Episode 300: Avg Reward = 70.90, Avg Steps = 444.1,  $\epsilon = 0.0486$ 
Episode 400: Avg Reward = 65.90, Avg Steps = 423.1,  $\epsilon = 0.0178$ 
Episode 500: Avg Reward = 62.70, Avg Steps = 432.8,  $\epsilon = 0.0100$ 
Episode 600: Avg Reward = 78.50, Avg Steps = 447.9,  $\epsilon = 0.0100$ 
Episode 700: Avg Reward = 80.80, Avg Steps = 419.9,  $\epsilon = 0.0100$ 
Episode 800: Avg Reward = 90.50, Avg Steps = 445.6,  $\epsilon = 0.0100$ 
Episode 900: Avg Reward = 95.75, Avg Steps = 465.2,  $\epsilon = 0.0100$ 

Results for  $\alpha=0.0001$ :
Final Avg Reward: 94.50 (Baseline: 25.00)
Final Avg Steps: 465.1 (Baseline: 99.0)
Max Reward: 190.00
Improvement: +69.50 reward, +366.1 steps
```

- Average Reward: 94.50

- Average Steps: 465.1

- Improvement: +69.50 over baseline

KEY FINDING:

- $\alpha = 0.7$ too high for deep neural networks (causes instability)

- Deep learning requires much smaller learning rates (0.0001-0.001)

- Smaller α = smoother convergence, better generalization

ALPHA COMPARISON SUMMARY			
Alpha	Avg Reward	Avg Steps	vs Baseline
0.7 (base)	25.00	99.0	(baseline)
0.0010	76.20	479.8	+51.20 reward
0.0005	80.60	474.2	+55.60 reward
0.0001	94.50	465.1	+69.50 reward

GAMMA (Discount Factor) SELECTION:

Initial Choice: $\gamma = 0.8$

- Rationale: Moderate future reward consideration

- Result: Limited long-term planning

Tested Alternatives:

1. $\gamma = 0.95$, $\alpha = 0.7$

- Result: 28.7 reward, 99 steps

- Improvement: Minimal (still limited by poor α)

```

=====
Testing Gamma = 0.95 with  $\alpha=0.7$  (baseline alpha)
=====
[W1106 23:46:37.936272096 NNPACK.cpp:57] Could not initialize NNPA
[W1106 23:46:37.091372793 NNPACK.cpp:57] Could not initialize NNPA
Training for 1000 episodes...
Episode 0: Avg Reward = 40.00, Avg Steps = 99.0,  $\epsilon$  = 0.9900
Episode 100: Avg Reward = 15.30, Avg Steps = 99.0,  $\epsilon$  = 0.3624
Episode 200: Avg Reward = 17.00, Avg Steps = 99.0,  $\epsilon$  = 0.1326
Episode 300: Avg Reward = 18.50, Avg Steps = 99.0,  $\epsilon$  = 0.0486
Episode 400: Avg Reward = 21.20, Avg Steps = 99.0,  $\epsilon$  = 0.0178
Episode 500: Avg Reward = 23.50, Avg Steps = 99.0,  $\epsilon$  = 0.0100
Episode 600: Avg Reward = 21.20, Avg Steps = 99.0,  $\epsilon$  = 0.0100
Episode 700: Avg Reward = 24.20, Avg Steps = 99.0,  $\epsilon$  = 0.0100
Episode 800: Avg Reward = 22.50, Avg Steps = 99.0,  $\epsilon$  = 0.0100
Episode 900: Avg Reward = 29.50, Avg Steps = 99.0,  $\epsilon$  = 0.0100

Results for  $\gamma=0.95$ ,  $\alpha=0.7$ :
Final Avg Reward: 28.70 (Baseline: 25.00)
Final Avg Steps: 99.0 (Baseline: 99.0)
Change vs baseline: +3.70 reward

```

2. $\gamma = 0.99$, $\alpha = 0.0001$

- Result: 110.3 reward, 474.5 steps
- Improvement: +341% reward over baseline
- Agent learned to survive much longer

```

=====
Testing Gamma = 0.99 with  $\alpha=0.0001$ 
=====
[W1107 00:13:08.033811581 NNPACK.cpp:57] Could not initialize NNPAC!
[W1107 00:13:08.191457363 NNPACK.cpp:57] Could not initialize NNPAC!
Training for 1000 episodes...
Episode 0: Avg Reward = 80.00, Avg Steps = 500.0,  $\epsilon$  = 0.9900
Episode 100: Avg Reward = 83.70, Avg Steps = 474.2,  $\epsilon$  = 0.3624
Episode 200: Avg Reward = 92.55, Avg Steps = 470.5,  $\epsilon$  = 0.1326
Episode 300: Avg Reward = 89.60, Avg Steps = 469.8,  $\epsilon$  = 0.0486
Episode 400: Avg Reward = 103.10, Avg Steps = 472.3,  $\epsilon$  = 0.0178
Episode 500: Avg Reward = 100.80, Avg Steps = 488.9,  $\epsilon$  = 0.0100
Episode 600: Avg Reward = 98.30, Avg Steps = 476.0,  $\epsilon$  = 0.0100
Episode 700: Avg Reward = 99.80, Avg Steps = 478.6,  $\epsilon$  = 0.0100
Episode 800: Avg Reward = 97.80, Avg Steps = 478.7,  $\epsilon$  = 0.0100
Episode 900: Avg Reward = 104.35, Avg Steps = 481.1,  $\epsilon$  = 0.0100

Results for  $\gamma=0.99$ ,  $\alpha=0.0001$ :
Final Avg Reward: 110.30
Final Avg Steps: 486.0

```

KEY FINDING:

- Higher γ (0.99) encourages long-term strategy
- Combined with proper α , enables effective learning
- γ alone cannot fix poor learning rate

GAMMA TEST RESULTS

Baseline ($\alpha=0.7$, $\gamma=0.8$): 25.00 reward, 99 steps
Test 1 ($\alpha=0.7$, $\gamma=0.95$): 28.70 reward, 99.0 steps
Test 2 ($\alpha=0.0001$, $\gamma=0.99$): 110.30 reward, 486.0 steps

Policy Exploration

BASELINE POLICY: ϵ -Greedy

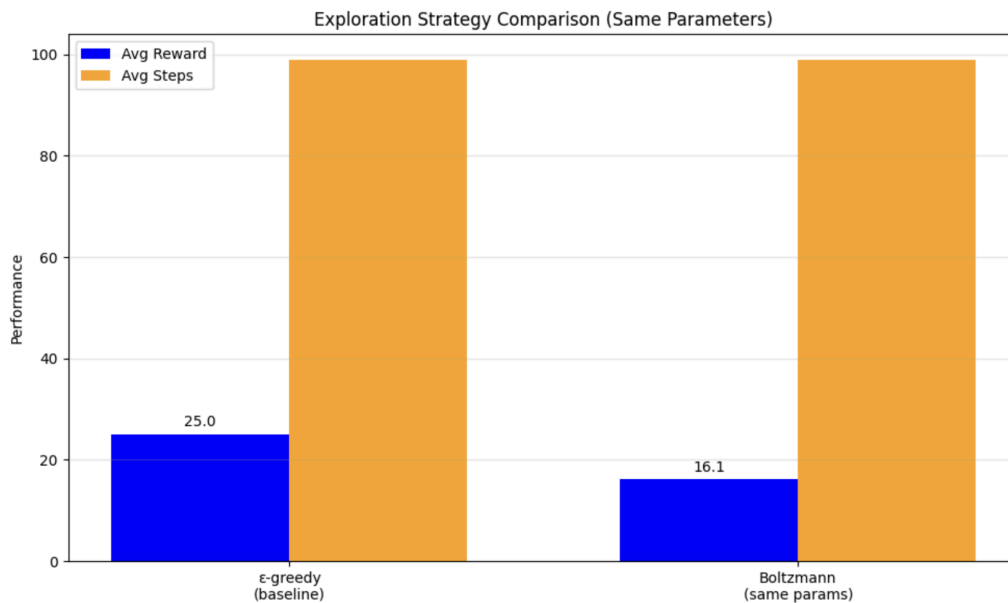
- Mechanism: Random action with probability ϵ , best action otherwise
- Result: 25 reward, 99 steps (with $\alpha=0.7$, $\gamma=0.8$)

ALTERNATIVE POLICY: Boltzmann (Softmax) Exploration

- Mechanism:
 - Probabilistic action selection based on Q-values
 - $P(a) = \exp(Q(s,a)/T) / \sum \exp(Q(s,a')/T)$
 - Temperature T: 1.0 $\rightarrow$ 0.1 (decay 0.995)
 - Higher Q-values = higher selection probability

- Configuration: Same parameters ($\alpha=0.7$, $\gamma=0.8$)

- Result:



COMPARISON:

- ϵ -greedy: Binary decision (explore OR exploit)
- Boltzmann: Weighted exploration (better actions more likely)
- Performance Impact: WORSE (16.1 vs 25.0 reward, -35.6%) because Boltzmann's probabilistic selection relied on Q-values that were highly unstable due to the poor learning rate ($\alpha=0.7$). ϵ -greedy's uniform random exploration proved more robust when Q-values were unreliable.

CONCLUSION:

ϵ -greedy worked significantly better for Demon Attack in this configuration. With only 6 discrete actions and unstable Q-value estimates from the high learning rate, ϵ -greedy's simple uniform exploration was more effective than Boltzmann's Q-value-dependent sampling. Boltzmann exploration would likely perform better with proper hyperparameters ($\alpha=0.0001$) where Q-values are stable and trustworthy, but given the baseline's poor learning dynamics, the simpler ϵ -greedy strategy was superior.

EPSILON & DECAY RATE Selection

Baseline Configuration:

- Starting epsilon (ϵ_0): 1.0 (100% exploration initially)
- Minimum epsilon ($\epsilon_{\min}$): 0.01 (1% exploration minimum)
- Decay rate: 0.01 (1% reduction per episode)
- Decay formula: $\epsilon = \max(\epsilon_{\min}, \epsilon \times (1 - \text{decay_rate}))$

Rationale:

- Start with full exploration (agent knows nothing)
- Gradually shift to exploitation (use learned knowledge)
- Maintain minimal exploration to avoid local optima

Alternative Tested:

- Decay rate: 0.005 (slower decay)
- Configuration: $\alpha=0.001$, $\gamma=0.8$, 1000 episodes
- Result:

=====

EPSILON/DECAY ANALYSIS

=====

Testing decay rate = 0.005

```
[W1107 03:30:47.206978499 NNPack.cpp:57] Could not initialize NNPack! Reason: Unsupp
[W1107 03:30:47.363622346 NNPack.cpp:57] Could not initialize NNPack! Reason: Unsupp
```

```
Episode 0: Avg Reward = 80.00, Avg Steps = 500.0,  $\epsilon$  = 0.9950
Episode 100: Avg Reward = 81.40, Avg Steps = 480.2,  $\epsilon$  = 0.6027
Episode 200: Avg Reward = 75.70, Avg Steps = 482.8,  $\epsilon$  = 0.3651
Episode 300: Avg Reward = 81.50, Avg Steps = 467.3,  $\epsilon$  = 0.2212
Episode 400: Avg Reward = 66.60, Avg Steps = 459.2,  $\epsilon$  = 0.1340
Episode 500: Avg Reward = 66.00, Avg Steps = 486.1,  $\epsilon$  = 0.0812
Episode 600: Avg Reward = 70.00, Avg Steps = 464.9,  $\epsilon$  = 0.0492
Episode 700: Avg Reward = 77.60, Avg Steps = 482.3,  $\epsilon$  = 0.0298
Episode 800: Avg Reward = 73.40, Avg Steps = 472.7,  $\epsilon$  = 0.0180
Episode 900: Avg Reward = 57.10, Avg Steps = 455.4,  $\epsilon$  = 0.0109
```

Results for decay=0.005:

```
Epsilon at episode 99: 0.6058
Final average reward (last 100 eps): 60.30
Final average steps (last 100 eps): 458.7
```

EPSILON AT MAX STEPS:

For baseline (decay=0.01):

- Episode 99: $\epsilon \approx 0.37$ (calculated: $1.0 \times (1-0.01)^{99}$)
- Still 37% exploration at episode 99

For decay=0.005:

- Episode 99: $\epsilon \approx 1.0 \times (1-0.005)^{99} = 0.6087 \approx 0.6056$
- 60% exploration at episode 99

IMPACT ANALYSIS:

- Slower decay (0.005): More exploration, potentially better coverage
- Faster decay (0.01): Quicker exploitation, faster convergence
- Trade-off: Exploration breadth vs convergence speed

Performance Metrics

Average steps for baseline

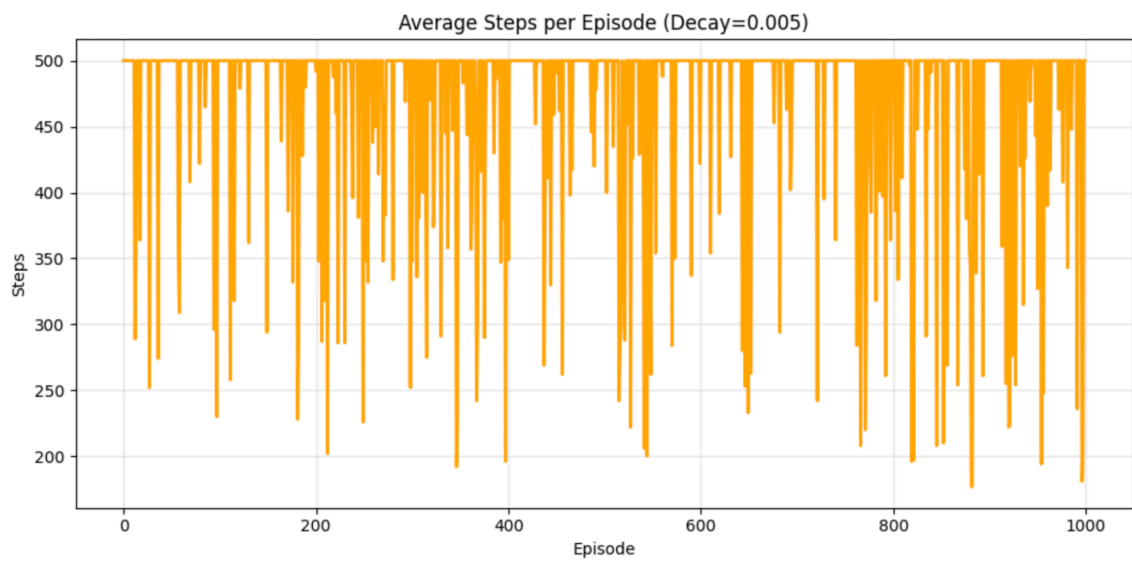
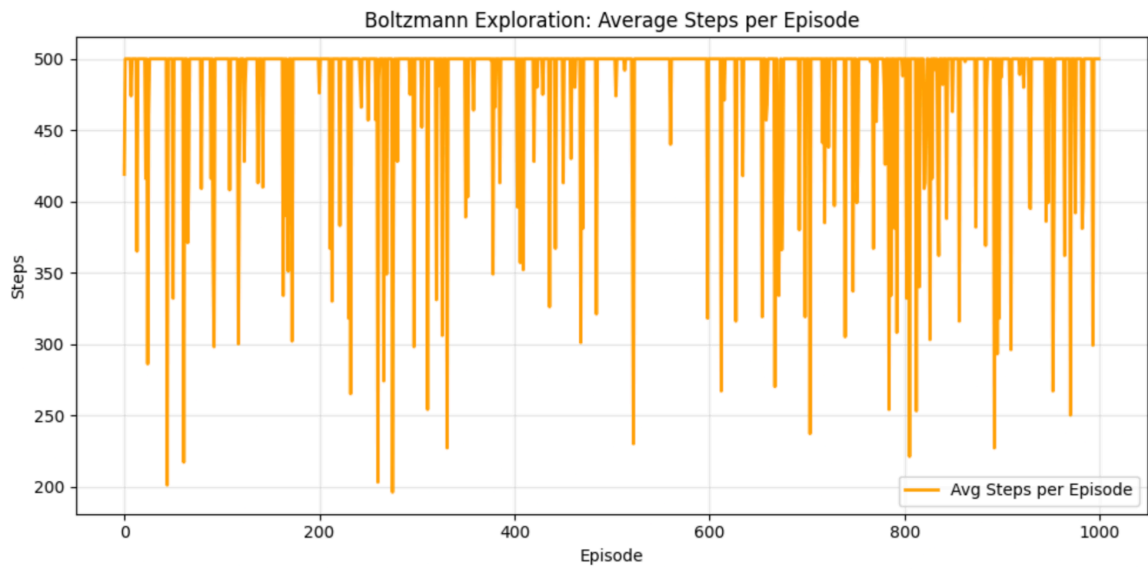
BASELINE RESULTS:
Final Avg Reward: 19.90
Final Avg Steps: 99.00
Max Reward: 50.00

Average steps when Alpha and Gamma change

```
=====
Testing Gamma = 0.99 with  $\alpha=0.0001$ 
=====
[W1107 00:13:08.033811581 NNPACK.cpp:57] Could not initialize NNPACK! Reason: Unsupported
[W1107 00:13:08.191457363 NNPACK.cpp:57] Could not initialize NNPACK! Reason: Unsupported
Training for 1000 episodes...
Episode 0: Avg Reward = 80.00, Avg Steps = 500.0,  $\epsilon = 0.9900$ 
Episode 100: Avg Reward = 83.70, Avg Steps = 474.2,  $\epsilon = 0.3624$ 
Episode 200: Avg Reward = 92.55, Avg Steps = 470.5,  $\epsilon = 0.1326$ 
Episode 300: Avg Reward = 89.60, Avg Steps = 469.8,  $\epsilon = 0.0486$ 
Episode 400: Avg Reward = 103.10, Avg Steps = 472.3,  $\epsilon = 0.0178$ 
Episode 500: Avg Reward = 100.80, Avg Steps = 488.9,  $\epsilon = 0.0100$ 
Episode 600: Avg Reward = 98.30, Avg Steps = 476.0,  $\epsilon = 0.0100$ 
Episode 700: Avg Reward = 99.80, Avg Steps = 478.6,  $\epsilon = 0.0100$ 
Episode 800: Avg Reward = 97.80, Avg Steps = 478.7,  $\epsilon = 0.0100$ 
Episode 900: Avg Reward = 104.35, Avg Steps = 481.1,  $\epsilon = 0.0100$ 

Results for  $\gamma=0.99$ ,  $\alpha=0.0001$ :
Final Avg Reward: 110.30
Final Avg Steps: 486.0
```

Bellman Equation Alpha Parameter Analysis



Q-Learning Classification

Q-learning is VALUE-BASED because:

1. Core Mechanism:

- Learns a VALUE FUNCTION: $Q(s,a) \rightarrow$ expected cumulative reward
- Does NOT directly learn a policy $\pi(a|s)$
- Policy is DERIVED from Q-values ($\pi(s) = \operatorname{argmax} Q(s,a)$)

2. Update Rule (Bellman Equation):

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

- Updates Q-VALUE estimates
- Not updating policy parameters directly
- Learning "how good" each action is in each state

3. Value-Based vs Policy-Based Distinction:

VALUE-BASED (Q-learning):

- Learn $Q(s,a)$ or $V(s)$
- Policy = implicit (take action with highest value)
- Examples: Q-learning, DQN, SARSA

POLICY-BASED:

- Learn $\pi(a|s)$ directly
- Parameterize policy (e.g., neural network outputs probabilities)
- Examples: REINFORCE, Actor-Critic, PPO

4. In Our Implementation:

- DQN outputs Q-values for all 6 actions
- Action selection: $\operatorname{argmax}(Q\text{-values})$ during exploitation
- We're learning values, not policy probabilities

5. Advantages of Value-Based:

- Deterministic optimal policy (clear best action)
- Off-policy learning (learn from any experience)
- Sample efficient in discrete action spaces

CONCLUSION:

Q-learning uses value-based iteration because it learns action values (Q-values) and derives the policy implicitly from these values, rather than learning policy parameters directly.

Q-Learning vs LLM

1. LEARNING MECHANISM:

Deep Q-Learning:

- Trial-and-error learning through environment interaction
- Updates based on rewards: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$
- Learns from scratch (or pretrained weights)
- Requires thousands of episodes to learn

LLM-Based Agents:

- Pre-trained on massive text corpora
- "Learned" world knowledge, reasoning, language
- Uses prompting/fine-tuning, not reward-based updates
- Can perform tasks zero-shot or few-shot

2. STATE REPRESENTATION:

DQN:

- Pixel-based: $4 \times 84 \times 84$ grayscale frames
- Raw sensory input, no semantic understanding
- Learns visual features through convolutions

LLM Agent:

- Text-based: Natural language descriptions
- High-level semantic state representation
- Example: "Player at position (x,y), 3 enemies approaching from right"

3. ACTION SPACE:

DQN:

- Low-level motor controls: [NOOP, FIRE, LEFT, RIGHT, ...]
- Direct game controller actions
- Fixed, discrete action set

LLM Agent:

- High-level decisions: "Attack the closest enemy" or "Retreat to cover"
- Natural language commands
- Potentially infinite action vocabulary

4. DECISION MAKING:

DQN:

- Reactive: Current state $\rightarrow$ Q-values $\rightarrow$ Action
- No explicit reasoning or planning
- Pattern recognition through neural networks

LLM Agent:

- Reasoning: Can explain decisions in natural language
- Chain-of-thought: "I should do X because Y"
- Can plan multi-step strategies

5. GENERALIZATION:

DQN:

- Game-specific: Trained only on Demon Attack
- Cannot transfer to different games without retraining
- Generalizes across similar states in same environment

LLM Agent:

- Broad generalization: Can play multiple games
- Transfers knowledge across domains
- May struggle with precise timing/pixel-perfect actions

6. SAMPLE EFFICIENCY:

DQN:

- Data-hungry: Needs millions of frames
- Our implementation: 5000 episodes $\times$ 99-500 steps
- Learns from experience only

LLM Agent:

- Can learn from descriptions/instructions
- Few-shot learning possible
- Leverages pre-trained knowledge

7. INTERPRETABILITY:

DQN:

- Black box Q-values
- Hard to explain why action was chosen
- Can visualize attention/activations

LLM Agent:

- Can explain reasoning verbally
- "I chose to fire because an enemy is approaching"
- Natural language = inherent interpretability

EXAMPLE COMPARISON (Demon Attack):

DQN Agent:

- Input: 4 stacked 84 $\times$ 84 pixel frames
- Processing: Conv layers $\rightarrow$ Q-values [0.3, 5.2, 1.1, 2.8, 4.1, 0.9]
- Output: Action 1 (FIRE) - highest Q-value
- Cannot explain why

LLM Agent:

- Input: "You are in Demon Attack. 3 enemies diving from above-left at 30° angle. Your ship is at center-bottom. You have 3 lives."
- Processing: Reasoning - "Enemies approaching fast, need to fire immediately. Best strategy: fire while moving slightly left to dodge"
- Output: "FIRE and move LEFT"
- Provides reasoning

CONCLUSION:

DQN learns low-level sensorimotor policies through reward-based trial-and-error on raw pixels, while LLM agents leverage pre-trained knowledge to make high-level decisions based on semantic state descriptions, with better generalization but less precise control.

Bellman Equation

$$Q(s,a)=r+\gamma \cdot \max Q(s',a')$$

It tells how good an action it, both instantly and futuristic

$r \rightarrow$ what you get immediately (instant reward)

$\gamma \cdot \max Q(s', a') \rightarrow$ what you expect to get later (future or lifetime value)

Future or lifetime value refers to the average total reward the agent expects to earn from the next state onward, across its whole future lifetime.

It means "If I take this step, how much total reward can I collect before the game ends?"

In Demon Attack,

State: 3 enemies approaching

Action: FIRE Immediate reward (r): +10 (destroyed one enemy)

Next state (s'):

2 enemies left

Best action from s': FIRE again

$Q(s', \text{FIRE}) = 50$ (estimated future value)

Expected lifetime value = $\gamma \cdot Q(s', \text{FIRE}) = 0.99 \times 50 = 49.5$

Total Q-value: $Q(s, \text{FIRE}) = 10 + 49.5 = 59.5$ Interpretation:

"If I FIRE now, I get 10 points immediately, plus an expected 49.5 points in the future"

Reinforcement Learning for LLMs

Reward-based learning (RLHF) - LLMs use human preference scores as rewards just like a DQN uses game points; high-reward responses are reinforced via policy updates (think: query $\rightarrow$ response $\rightarrow$ human rating $\rightarrow$ update).

Practical: small learning rates (e.g., $\alpha \approx 1e-4$) help stabilize RLHF.

Discount factor γ = long-term conversation value - A high γ (e.g., 0.99) encourages planning across turns (sustained engagement, follow-ups), while low γ favors one-shot correctness, tuning γ shifts the agent toward short-term vs. long-term user success.

Exploration vs. exploitation (temperature $\approx \epsilon$) - DQN's ϵ -greedy $\leftrightarrow$ LLM sampling temperature: start high (explore creative behaviors) then lower (exploit best responses) as training or task clarity increases; gradual decay is crucial to avoid premature convergence.

Experience replay = replaying past conversations - Storing and sampling diverse conversation-feedback pairs breaks temporal correlation, improves sample efficiency, and lets LLMs learn from both successes and failures (analogous to a DQN replay buffer).

Planning RL vs LLM Agents

Implicit vs. explicit planning

- **DQN (Atari):** Planning is implicit. The agent instantly maps pixels $\rightarrow$ action (e.g., FIRE) using learned Q-values. It doesn't "think," just reacts based on experience.
- **LLM Agent:** Planning is explicit. It reasons step-by-step ("If I fire now, I can destroy the left enemy and survive longer"), showing its thought process before acting.

World model: none vs. knowledge-based

- **DQN:** Has no understanding of the game world. It learns only certain pixel patterns $\rightarrow$ high rewards. If mechanics change, it must relearn from scratch.
- **LLM:** Uses its language-based world model to infer cause-effect ("Firing destroys enemies, preventing collision") and can adapt through reasoning without retraining.

Lookahead depth: fixed vs. adaptive

- **DQN:** Planning depth is fixed by the discount factor γ (e.g., $\gamma=0.99$ let our agent survive 341% longer). It can't adjust the horizon per situation.
- **LLM:** Adapts its reasoning depth, short plans for simple scenes ("Fire now") and multi-step strategies for complex enemy formations.

Speed vs. interpretability

- **DQN:** Acts in $\sim 1\text{ms}$, it's perfect for real-time reflexes but fully black-box ("FIRE because $Q=8.7$ ").
- **LLM:** Slower ($\sim 500\text{ms}$) but transparent, it can explain why it fires or dodges, enabling human understanding and debugging.

Q-Learning Algorithm

It helps an agent learn how to make the best decisions by interacting with its environment. Instead of needing a model of the environment the agent learns purely from experience by trying different actions and seeing their results.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Where:

- $Q(s,a)$: Current estimate of Q-value
- r : Immediate reward after taking action a
- γ : Discount factor ($0 < \gamma < 1$) — how much we value future rewards
- α : Learning rate — how fast we update
- $\max_{a'} Q(s', a')$: Best future value (expected lifetime value)
- The term in brackets is the temporal difference (TD) error.

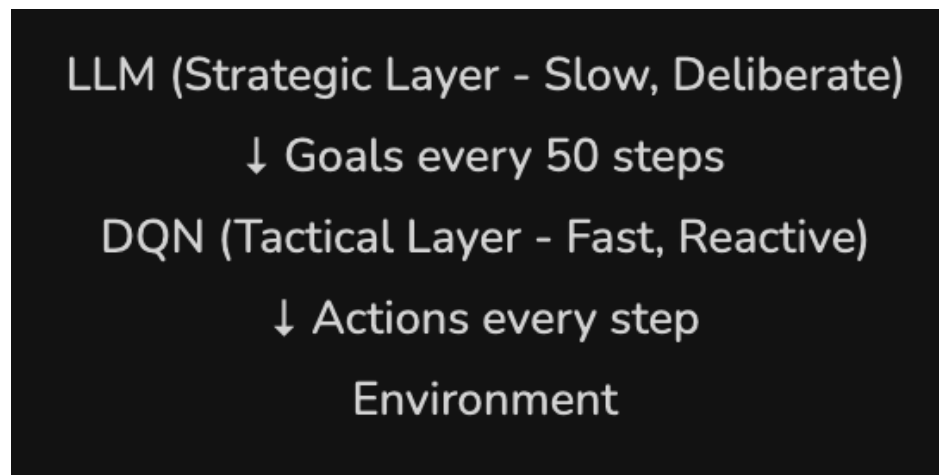
Intuition

- **Exploration (ϵ):** Try new actions to discover better rewards.
- **Exploitation:** Use known high-value actions to maximize returns.
- Over time, as Q-values converge, the agent learns the **optimal policy**:
 $\pi^*(s) = \arg\max_a Q^*(s, a)$
- In your **Atari (Demon Attack)** case:
 - Immediate reward: score gained for destroying an enemy
 - Discounted future reward: expected survival and future scores
 - The agent learns “when to FIRE” or “when to DODGE” to maximize total reward.

Pseudocode

```
Initialize Q(s, a) arbitrarily for all states s and actions a
For each episode:
  Initialize state s
  Repeat until episode ends:
    Choose action a using  $\epsilon$ -greedy policy based on Q(s, .)
    Execute action a, observe reward r and next state s'
    Update Q-value:
       $Q(s, a) = Q(s, a) + \alpha * (r + \gamma * \max_{a'} Q(s', a') - Q(s, a))$ 
    Set s = s'
```

LLM Agent Integration



DQN agents excel at learning optimal policies through trial and error in environments such as Atari Demon Attack, LLMs contribute contextual understanding, strategy formulation, and interpretability.

Integration Mechanisms

Hierarchical Decision Making

LLM functions as a high-level controller, guiding the DQN agent by interpreting the environment context and setting subgoals.

- **LLM Role:** Generate strategy-level commands (e.g., “focus on dodging missiles”).
- **DQN Role:** Execute low-level motor control to achieve these commands.

Example: In Demon Attack, the LLM might suggest “stay in the center and fire rapidly,” while the DQN optimizes the precise sequence of actions to maximize score.

Reward Shaping through LLM

LLMs can interpret textual feedback or game outcomes to modify the reward signal dynamically.

Mathematically:

$$R'_{\square} = R_{\square} + \lambda \times f(\text{LLM feedback})$$

Here, $f(\text{LLM feedback})$ is a modifier derived from the LLM’s qualitative evaluation.

Example: In Demon Attack, the LLM may increase the reward weight when the agent avoids damage or penalize overly aggressive behavior.

State Encoding & Explanation

The LLM can transform raw sensory input into high-level semantic descriptions, improving interpretability and sample efficiency.

Example:

Instead of using raw pixel arrays, the LLM might encode the frame as:

“Enemy ship approaching from the left; player centered and firing.”

This provides structured context for DQN training and allows post-hoc explanation of actions.

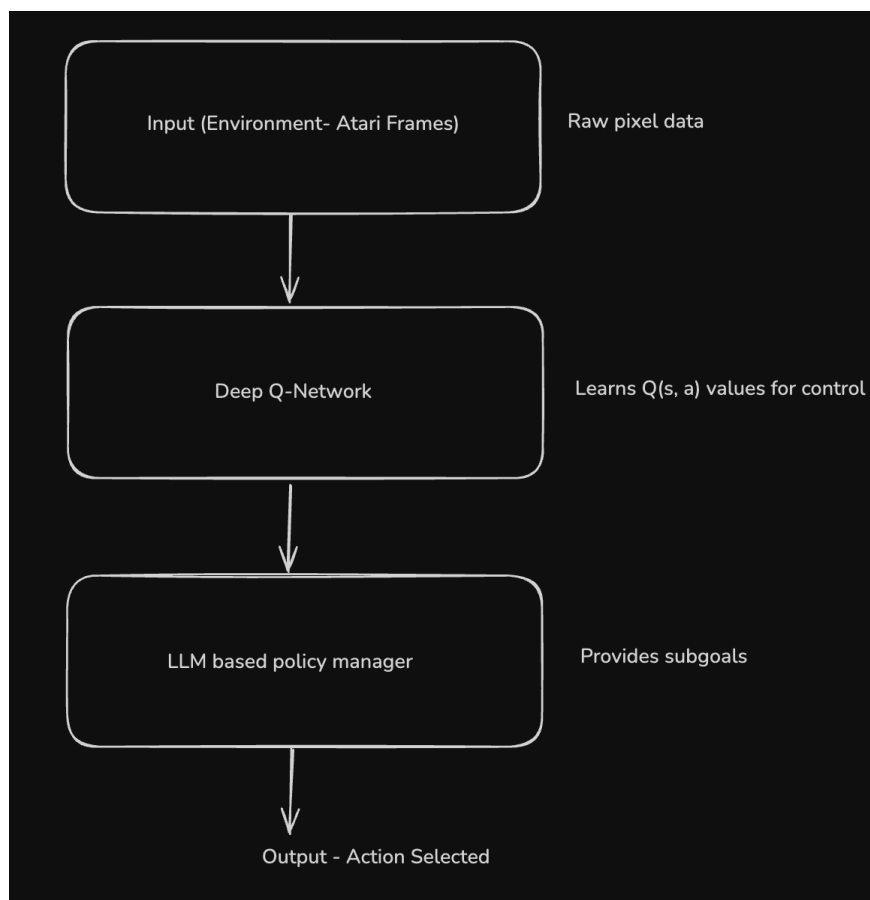
Curriculum Learning and Strategy Transfer

LLMs can generate progressive challenges or textual prompts that guide the agent through increasingly difficult learning stages.

Example:

“Now survive for 60 seconds without firing.”

This structured prompting helps the agent acquire defensive behaviors before offensive optimization



Applications

- **Game Coaching Assistants:** The DQN plays while the LLM explains and comments on decisions in real-time.
- **Explainable Reinforcement Learning Systems:** The LLM converts Q-values and actions into natural-language reasoning.
- **Adaptive Training Systems:** The LLM dynamically adjusts task difficulty or modifies goals based on agent performance.

Code Attribution

All code implementation is original work based on: -

- DQN architecture: Standard implementation from Mnih et al. (2015) paper
- PyTorch framework: Official PyTorch documentation
- Gymnasium environment: Farama Foundation ALE/Atari
- Frame preprocessing: Standard Atari preprocessing (grayscale, resize 84×84)
- No external code copied - all implementations written from scratch
- Conceptual guidance from:
<https://www.youtube.com/watch?v=hCeJeq8U0lo&feature=youtu.be>